Skinning that 'droid!

Show your Android OS who's boss by completely skinning it to your taste. It's not as hard as you would imagine.

Abhijit Dey
abhijit.dey@9dot9.in

You may have come across many custom interfaces on Android that not only enhance user interface design, but also add more functionality, making it easier to access certain features and options. Such themes or interfaces are provided right out of the box by many manufacturers such as TouchWiz, Sense, MIUI, LG UI, Xperia UI, etc. or as downloadable options in certain custom Android ROMs out there such as Cyanogenmod. Wouldn't it be great if you could design or build your own Android theme that perfectly fits your requirements and interests? If you really believe that, read on.

This brief tutorial endeavours to introduce you to the process of theming and building your own Android theme. The tutorial is independant of device, but assumes that you're using a Windows PC.

Getting Started

First and foremost, there are certain prerequisites to theming your own custom skin. You'll obviously need a rooted phone since accessing the framework and system files are crucial for making key modifications. If you are planning on rooting your Android phone, here's a definitive guide on the same http://dgit.in/AndroidRootDT. Be advised: rooting might void your warranty on certain smartphones, but some phones can be unrooted cleanly without voiding warranty.

Installing Java SE Runtime Environment

Download the latest version of the Java SE Runtime Environment from http://dgit.in/JavaSERun if you haven't already, and ensure that the right version and operating system is selected. Open "System Properties" from "Control Panel" and go to the "Environment Variables" section.

You now need to specify the Java installation path on your Windows drive in the "System variables" section in order to run Java tools. Select the "Path" variable in "System variables" and click on "Edit" to enter the Java directory path (don't forget to add a semi-colon before the directory path). To verify the functionality of Java commands, open up "Command Prompt" and enter `java -version` to display the installed version. If you don't encounter any errors, you may proceed.

Setting up the Android SDK

Download and install "Android SDK" from http://dgit.in/AndroidSDKIn. You can either download the Android SDK bundle with "Android Studio" included but you will be fine with just the "Stand-alone SDK Tools" for this tutorial. After downloading Android SDK, install a few packages from "Android SDK Manager". The important ones are: "Android SDK Tools", "Android SDK Platform-tools" and "Android SDK Build-tools". Sometimes, your phone might not be able to push the drivers on to your PC, so you may also need to install the Google USB Driver package from Android SDK Manager.

When you're finished installing all necessary packages, connect your phone to your computer. Before connecting your phone, make sure that "USB Debugging" is switched on. This can be accessed from "Developer Options", which is hidden by default. To enable that, you need to go to `Settings > About Phone` and tap seven times on "Build Number".

You will now be able to access the Developer Options of your device. Now open "Device Manager" on your PC to verify whether you can find a device named "Android Phone > Android Composite ADB Interface". The phone has failed to install the drivers on your PC if the device is shown under "`Other Devices > Unknown Device`".

You can download the Universal ADB Driver from http://dgit.in/UnivADB which works most of the time. To ensure driver installation, right-click on "Unknown Device" in Device Manager and select "Uninstall". While your device is connected, click on the "Scan for hardware changes" button at the top, which will list with your device again. If this still doesn't display your device, right-click again on Unknown Device and select the "Update